Практикум по документации

Руководство оператора и проверка запуска по инструкции

За одну практическую пару вы подготовите руководство для знакомой программы проверки конфигурации, сохраните документ в Git и проверите запуск из отдельной копии выпуска. Программа и примеры ошибок уже включены в стартовый архив.

Что должно получиться

Руководство docs/operator-guide.txt с четырьмя заполненными разделами; протокол REVIEW.txt; наблюдения одного успешного запуска и трёх отказов; проверенный выпуск и история Git. Материалы собираются в один ZIP для последующей загрузки в ЭОС.

Работа	Время
Подготовить папку и среду	8 минут
Изучить файлы и наблюдать результаты	12 минут
Написать руководство	18 минут
Сохранить редакцию в Git	7 минут
Проверить руководство по свежей копии	10 минут
Собрать и сохранить результат	5 минут

До практики преподаватель объясняет маршрут по презентации 25 минут. После самостоятельной работы предусмотрено 5 минут обсуждения. Ссылку и срок сдачи в ЭОС преподаватель объявит отдельно. Сейчас сохраняйте результаты локально.

Как читать методичку

Страницы 2–14 образуют последовательный маршрут. Страница 15 помогает устранить ошибки, страница 16 содержит критерии и дополнительное задание. Иллюстрации терминала подготовлены по фактическому проверочному запуску. Это учебные изображения вывода, а не снимки рабочего стола Windows. Путь, имя автора и хеши на вашем компьютере будут другими.

1 Подготовка Windows

Скачайте 03_Стартовый_проект.zip. Нажмите по нему правой кнопкой и выберите «Извлечь всё». Откройте получившуюся папку operator-lab. Используйте новую папку, чтобы не смешивать текущую работу с прежними упражнениями. Внутри должны быть check.sh, config.txt, README.txt, папка docs и вспомогательные файлы.

Git for Windows устанавливается один раз. Если он уже использовался на прошлых занятиях, повторная установка не нужна. При отсутствии Git используйте официальный адрес https://git-scm.com/install/windows и после установки заново откройте терминал. Интернет необходим для получения установщика; само упражнение работает локально.

Где вводить команды

Дважды щёлкните 00_OPEN_GIT_BASH.cmd. Резерв: в этой папке откройте контекстное меню и выберите Open Git Bash here. В Windows 11 сначала может понадобиться «Показать дополнительные параметры». Вводите команды в Git Bash по одной строке и нажимайте Enter. Для вставки используйте Shift+Insert или меню терминала.

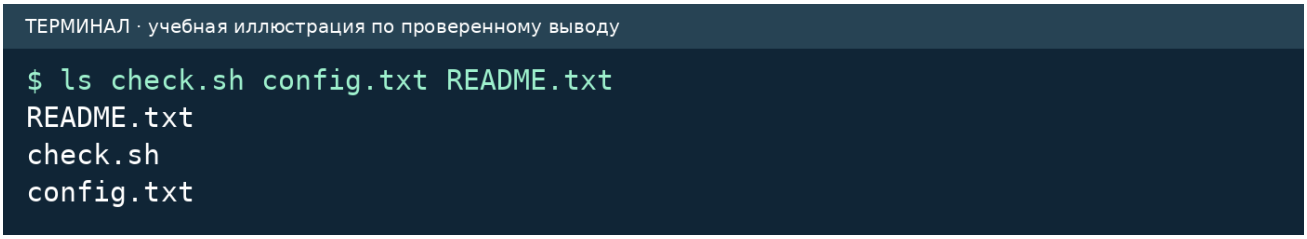


Рисунок 1. Проверка наличия трёх основных файлов. Порядок вывода имён несущественен.

```
ls check.sh config.txt README.txt
```

Символ $ в начале строки на иллюстрации обозначает приглашение и не вводится. В выражении echo $? знак $ нужен. Длинная строка на изображении может переноситься визуально; копируйте целую команду из текстового блока под рисунком.

Включите отображение расширений в Проводнике: в Windows 10 через вкладку «Вид», в Windows 11 через меню «Вид» и «Показать». Руководство будете редактировать в Блокноте, а команды выполнять в Git Bash.

2 Начальная подготовка

Проверьте папку командой pwd. Последняя часть пути должна быть operator-lab. Затем один раз выполните init.sh. Помощник создаёт репозиторий с веткой main, исходным коммитом и тегом v1.0.0. Для протоколов создаётся папка evidence. Он не подключается к удалённому серверу.

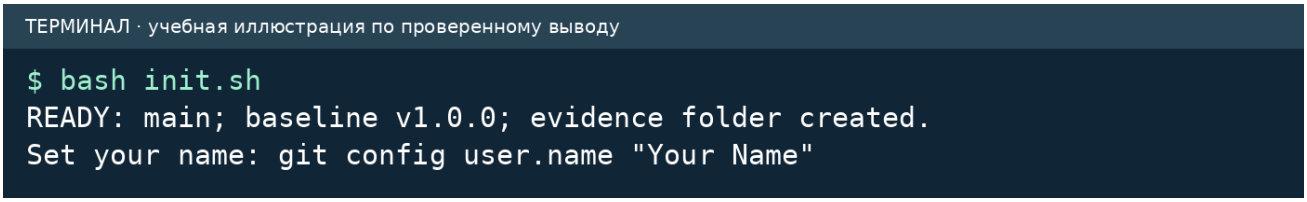


Рисунок 2. Успешная подготовка новой рабочей папки.

```
bash init.sh
```

После READY задайте своё имя автора. Адрес ниже является учебным техническим значением, реальная почта не требуется. Настройки относятся только к этому репозиторию.

```
git config user.name "Ваше Имя"
git config user.email "student@example.invalid"
git status --short
```

Пустой вывод последней команды означает, что отслеживаемые файлы пока не изменились. Если вместо READY появляется STOP о существующем репозитории, подготовка уже выполнена. Не удаляйте .git: продолжите со следующей страницы.

Если не получилось

Сообщение No such file or directory для init.sh обычно означает неверную папку либо неполную распаковку. Вернитесь в Проводник, откройте operator-lab и заново запустите помощник Git Bash. Сообщение о недостающей команде означает, что используется неподходящий терминал или неполная установка Git for Windows.

Если Git сейчас установить невозможно, прочитайте примеры и подготовьте черновик руководства. Не отмечайте технические проверки как выполненные: их необходимо завершить после подготовки среды.

3 Что делает программа

Перед описанием программы изучите её входные файлы. Команда cat выводит их содержимое в терминал. Первые две строки относятся к config.txt, следующая — к README.txt. Показанный интервал равен 60, число повторов — 3.

ТЕРМИНАЛ · учебная иллюстрация по проверенному выводу

```
$ cat config.txt README.txt
interval=60
retries=3
Monitoring interval: 60 seconds.
```

Рисунок 3. Исходные настройки и строка описания согласованы.

`cat config.txt README.txt`

Файл	Назначение
check.sh	Проверяет наличие файлов, допустимые значения и строку описания.
config.txt	Содержит interval=60 и retries=3.
README.txt	Содержит описание принятого интервала.
docs/operator-guide.txt	Ваше руководство для пользователя.
cases	Готовые неправильные примеры для наблюдения отказов.

В программе разрешены interval=30 либо interval=60 и retries=3. Для каждого интервала README должен содержать соответствующую строку. В исходной версии принято 60. Скрипт проверяет файлы и завершает работу. Он не запускает настоящий мониторинг, не измеряет нагрузку и не выполняет сетевые запросы.

Небольшое исследование

Откройте check.sh в Блокноте только для чтения. Найдите слова fail, required file missing и README and config disagree. Соотнесите их с тремя видами отказов на следующих страницах. Не редактируйте скрипт для получения положительного ответа.

4 Успешный запуск и код

Выполните три строки подряд. Первая запускает программу и сохраняет оба потока сообщений в evidence/01_ok.txt. Вторая записывает код завершения первой команды. Третья показывает сохранённые результаты. Все действия выполняются из operator-lab.

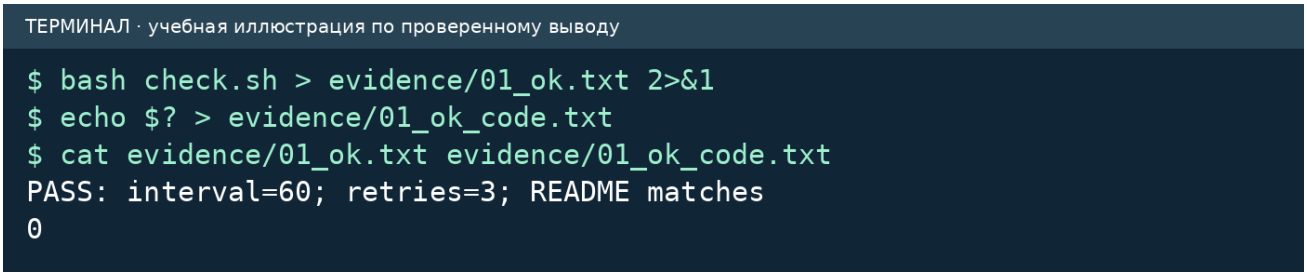


Рисунок 4. Успешная проверка и код завершения 0.

```
bash check.sh > evidence/01_ok.txt 2>&1
echo $? > evidence/01_ok_code.txt
cat evidence/01_ok.txt evidence/01_ok_code.txt
```

Ожидается PASS: interval=60; retries=3; README matches и затем число 0. Сообщение означает, что проверяемые правила соблюдены. Это не заключение о работоспособности настоящего сервера или всех возможных свойствах приложения.

Почему порядок строк важен

Выражение $? содержит код последней выполненной команды. Между запуском проверки и echo $? нельзя вводить pwd, cat или другую команду. Если вы это сделали, повторите все три строки. Знак > создаёт или заменяет только указанный протокол. Конструкция 2>&1 отправляет туда и сообщения ошибок.

Если после первой строки ничего не видно, это ожидаемо: вывод направлен в файл. Упражнение не требует ожидания фонового процесса. После завершения команды терминал снова показывает приглашение.

Что перенести в руководство

Запишите папку запуска, команду bash check.sh, точное успешное сообщение и способ проверки кода. Укажите, что программа завершается сама. Команды перенаправления нужны для отчёта студента; в обычном пользовательском запуске достаточно bash check.sh и сразу echo $?.

5 Два ожидаемых отказа

Сначала укажите отсутствующий файл absent.txt. Это специально неверное имя. Основной config.txt остаётся на месте. Выполните строки в указанном порядке.

ТЕРМИНАЛ · учебная иллюстрация по проверенному выводу

```
$ bash check.sh absent.txt README.txt > evidence/02_missing.txt 2>&1
$ echo $? > evidence/02_missing_code.txt
$ cat evidence/02_missing.txt evidence/02_missing_code.txt
FAIL: required file missing
1
```

Рисунок 5. Отсутствующий входной файл: отказ и код 1.

```
bash check.sh absent.txt README.txt > evidence/02_missing.txt 2>&1
echo $? > evidence/02_missing_code.txt
cat evidence/02_missing.txt evidence/02_missing_code.txt
```

Затем используйте подготовленный файл cases/invalid.txt с interval=0. Условия упражнения разрешают только 30 либо 60. Эта команда не изменяет исходную конфигурацию.

ТЕРМИНАЛ · учебная иллюстрация по проверенному выводу

```
$ bash check.sh cases/invalid.txt README.txt > evidence/03_invalid.txt
2>&1
$ echo $? > evidence/03_invalid_code.txt
$ cat evidence/03_invalid.txt evidence/03_invalid_code.txt
FAIL: config must contain interval=30 or 60 and retries=3
1
```

Рисунок 6. Неподдерживаемая конфигурация: другой текст отказа и тот же код 1.

```
bash check.sh cases/invalid.txt README.txt > evidence/03_invalid.txt 2>&1
echo $? > evidence/03_invalid_code.txt
cat evidence/03_invalid.txt evidence/03_invalid_code.txt
```

Для руководства сформулируйте разные действия: при отсутствии файла проверить путь и наличие; при неверных значениях сверить данные с согласованной конфигурацией. Не предлагайте переустановку Git как универсальное решение.

6 Рассогласование описания

В cases/config30.txt записан допустимый интервал 30. Но исходный README описывает 60. Поэтому проверка должна завершиться отказом, хотя само значение интервала разрешено. Выполните команды и сохраните результат.

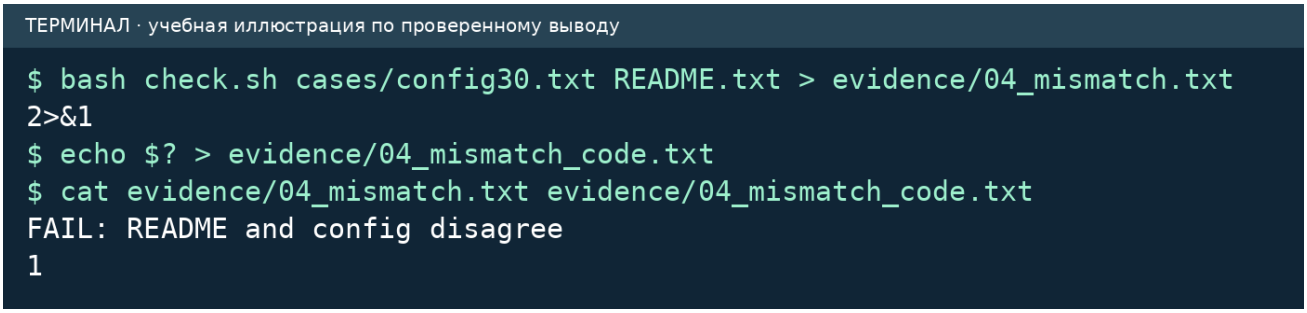


Рисунок 7. Допустимая настройка не совпала с её описанием.

```
bash check.sh cases/config30.txt README.txt > evidence/04_mismatch.txt 2>&1
echo $? > evidence/04_mismatch_code.txt
cat evidence/04_mismatch.txt evidence/04_mismatch_code.txt
```

Как объяснить причину

Сообщение README and config disagree означает расхождение между настройкой и документом. Код 1 сообщает о неуспехе; точная причина определяется текстом. Для этого упражнения основной выпуск сохраняет interval=60. Файл с 30 используется только как пример отказа.

В реальном сопровождении сначала нужно установить согласованное значение. Затем исправляют настройку или описание и повторяют проверку. Простая замена текста ради PASS может скрыть ошибку требований. В этой работе редактировать config.txt и README.txt не нужно.

Промежуточный результат

В evidence должны появиться четыре пары файлов: 01_ok, 02_missing, 03_invalid, 04_mismatch, у каждой — сообщение и код. Нормальный запуск имеет код 0, три специально неверных примера — код 1. При случайно сохранённом другом коде повторите соответствующий блок целиком.

Объясните своими словами: чем отличаются неправильное значение параметра и неправильное описание правильного значения? Ответ используйте при заполнении раздела сообщений оператору.

7 Черновик руководства

В Проводнике откройте docs/operator-guide.txt через Блокнот. Если расширение скрыто, сначала включите его отображение. Не создавайте новый файл рядом с шаблоном: заменяйте подсказки в существующем документе и сохраняйте в UTF-8.

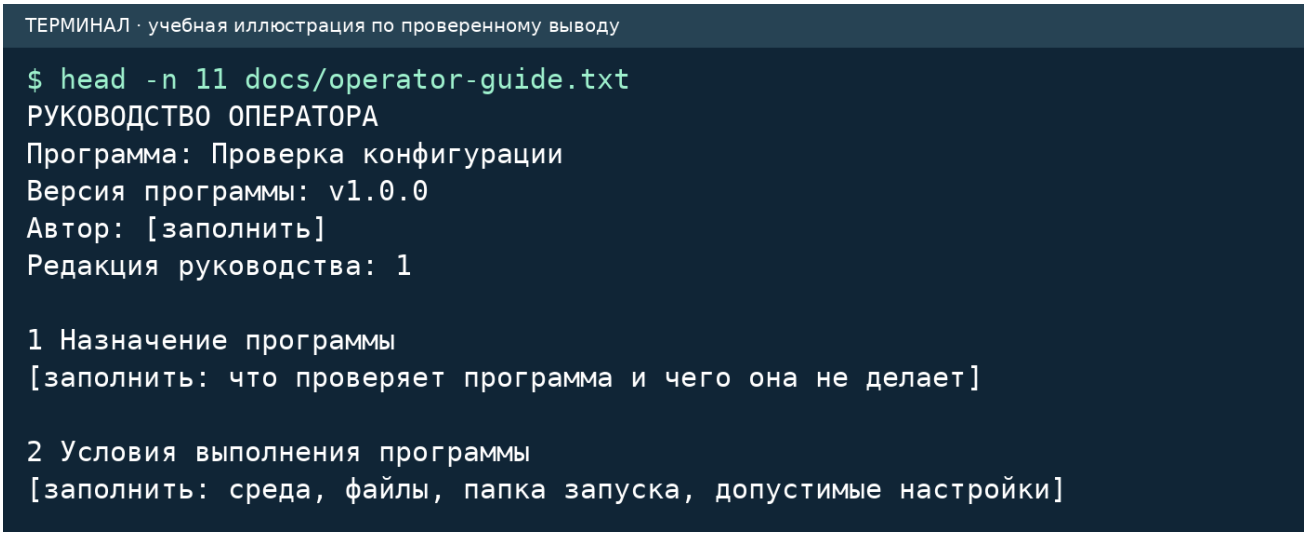


Рисунок 8. Начало шаблона до заполнения. Подсказки нужно заменить собственным текстом.

`head -n 11 docs/operator-guide.txt`

Сохраните четыре заголовка разделов и сведения о версии программы. В поле автора укажите себя. В назначении опишите проверку файлов и её границы. В условиях перечислите Git Bash, необходимые файлы, папку запуска и допустимые значения.

В выполнении программы дайте последовательные действия от распаковки до появления результата. В сообщениях разберите PASS и три FAIL, которые вы наблюдали. К каждому отказу добавьте причину и понятное действие. Не оставляйте в готовом документе подсказки вида [заполнить].

Достаточно примерно 1–2 страниц содержательного текста при печати. Не увеличивайте объём за счёт истории Git или общего описания компьютера. Полная методичка помогает выполнить задание, а ваше руководство должно помогать запустить конкретную программу.

8 Как писать понятные действия

Пользователь должен восстановить порядок действий из текста. Для каждого важного шага укажите исходную папку, действие и наблюдаемый результат. Скриншот помогает узнать окно, но точная команда должна остаться доступной текстом.

Неясная формулировка	Что нужно уточнить
Запустите программу	Откройте Git Bash в папке с check.sh. Введите bash check.sh и нажмите Enter.
Всё должно работать	Ожидается точное сообщение PASS, затем код 0 и новое приглашение терминала.
Исправьте ошибку	Назовите сообщение, проверяемый файл и условие, которому он должен соответствовать.
Остановите программу	Укажите, что проверка завершается автоматически и не создаёт постоянный процесс.

Готовый образец одного шага

«В распакованной папке выпуска найдите check.sh. Откройте Git Bash в этой папке. Введите bash check.sh. Для исходной настройки ожидается PASS: interval=60; retries=3; README matches. Сразу введите echo $?; ожидается 0. После завершения проверки снова появится приглашение терминала».

Это образец отдельного фрагмента. Остальные разделы заполните по наблюдениям. Не переписывайте эталон преподавателя: в студенческий комплект он не включён.

Проверка собственного текста до коммита

Прочитайте руководство как новый пользователь. Где находится папка? Какое окно нужно открыть? Что вводится буквально? Как выглядит успех? Что делать при каждом отказе? На каждый вопрос должен быть конкретный ответ. Проверьте, что слово «версия» относится к исходному коду v1.0.0, а редакция руководства описана отдельно.

В этой работе используется учебная структура из темы лекции. Формальное соответствие всему комплексу требований ЕСПД не заявляется и не оценивается.

9 Сохранение руководства в Git

Сохраните изменения в Блокноте. Затем укажите своё имя и добавьте только руководство. Вместо Student Example в первой строке введите своё имя. Остальные команды выполните без изменений.

ТЕРМИНАЛ · учебная иллюстрация по проверенному выводу

```
$ git config user.name "Student Example"
$ git add docs/operator-guide.txt
$ git diff --cached --stat
 docs/operator-guide.txt | 30 ++++++++++++++++++++++++++-----
 1 file changed, 25 insertions(+), 5 deletions(-)
```

Рисунок 9. В подготовленных изменениях должен быть файл руководства.

```
git config user.name "Student Example"
git add docs/operator-guide.txt
git diff --cached --stat
```

Создайте коммит и убедитесь, что больше нет отслеживаемых изменений. Сообщение коммита описывает добавление руководства. Хеш и число строк в вашем результате будут другими.

ТЕРМИНАЛ · учебная иллюстрация по проверенному выводу

```
$ git commit -m "docs: add operator guide"
[main 91587ea] docs: add operator guide
 1 file changed, 25 insertions(+), 5 deletions(-)
$ git status --short
```

Рисунок 10. Коммит документации и пустой вывод git status --short.

```
git commit -m "docs: add operator guide"
git status --short
```

Если статус показывает изменение check.sh, config.txt или README.txt, остановитесь и выясните причину. Для этой работы нужны изменения документации. Не добавляйте все файлы автоматически и не удаляйте рабочую папку при ошибке.

10 Свежая копия выпуска

После сохранения коммита выполните verify.sh. Он создаст архив файлов HEAD и отдельный каталог verification/run_1. Затем запустит программу из этой копии. В архиве нет истории .git. При повторном выполнении создаётся следующий каталог run_2, run_3 и так далее.

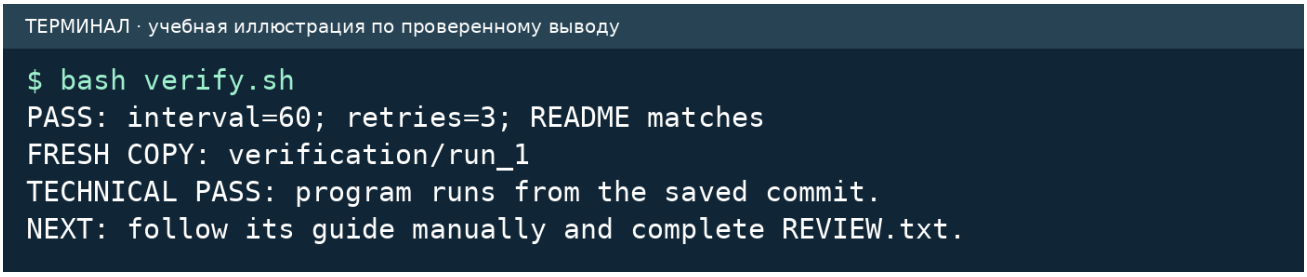


Рисунок 11. Технический запуск из свежей копии сохранённого состояния.

```
bash verify.sh
```

Ожидаются PASS и TECHNICAL PASS. Путь после FRESH COPY запишите в REVIEW.txt. Полный идентификатор коммита можно прочитать командой ниже. Его удобно скопировать в протокол без перепечатывания.

```
cat evidence/verified_commit.txt
```

Что подтверждено и что ещё предстоит

Помощник подтвердил, что программа запускается из файлов сохранённого коммита. Он не проверял, понятны ли человеку ваши русские предложения. Теперь нужна отдельная проверка именно текста руководства. Откройте docs/operator-guide.txt внутри указанной свежей копии и выполняйте её инструкции.

Если помощник просит создать коммит, проверьте git status --short. Если он сообщает о незаполненном руководстве, найдите оставшиеся подсказки. Если исходный код или конфигурация изменены, верните согласованное исходное содержимое после проверки своей ошибки; не обходите ограничение редактированием помощника.

11 Проверка по инструкции

Выполните самостоятельную проверку своего руководства. Используйте свежую копию, которую создал verify.sh. Откройте именно вложенный в неё docs/operator-guide.txt. Следуйте записанным действиям, не дополняя их по памяти. Если приходится угадывать папку или команду, отметьте пропуск для исправления.

Пункт чек-листа	Что зафиксировать
Подготовка	Папка запуска и наличие всех нужных файлов указаны в тексте.
Запуск	Команда введена именно по руководству, без догадок.
Результат	Получены указанное сообщение и код завершения.
Завершение и ошибки	Понятны завершение программы и действия при трёх FAIL.

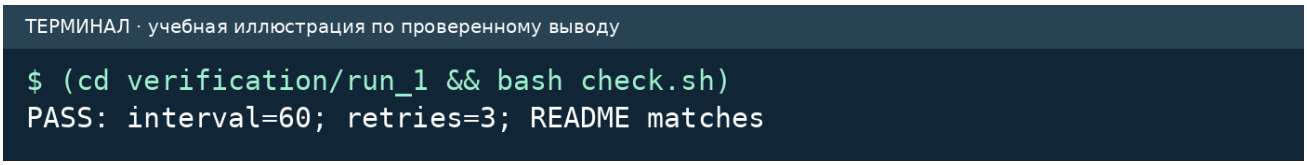


Рисунок 12. Технический пример повторного запуска из run_1. Он не заменяет чтение руководства.

`(cd verification/run_1 && bash check.sh)`

При второй и следующих попытках замените run_1 на каталог из сообщения verify.sh. Скобки в команде запускают действия во вложенной оболочке: после неё основной терминал остаётся в operator-lab. В протокол заносите фактический результат, а не ожидаемый ответ со страницы.

Важно: этот технический пример дан для сравнения. При содержательной проверке сначала прочитайте руководство свежей копии, затем повторите его действия. Запишите хотя бы одно конкретное наблюдение: найденный пропуск или понятный шаг, который удалось выполнить без дополнительных догадок.

12 Исправление замечаний

Откройте исходный docs/operator-guide.txt в operator-lab, а не его копию в verification. Исправьте конкретную неточность и сохраните файл. Если инструкция уже позволила пройти все действия, придумывать замечание и новый коммит не нужно: переходите к заполнению протокола.

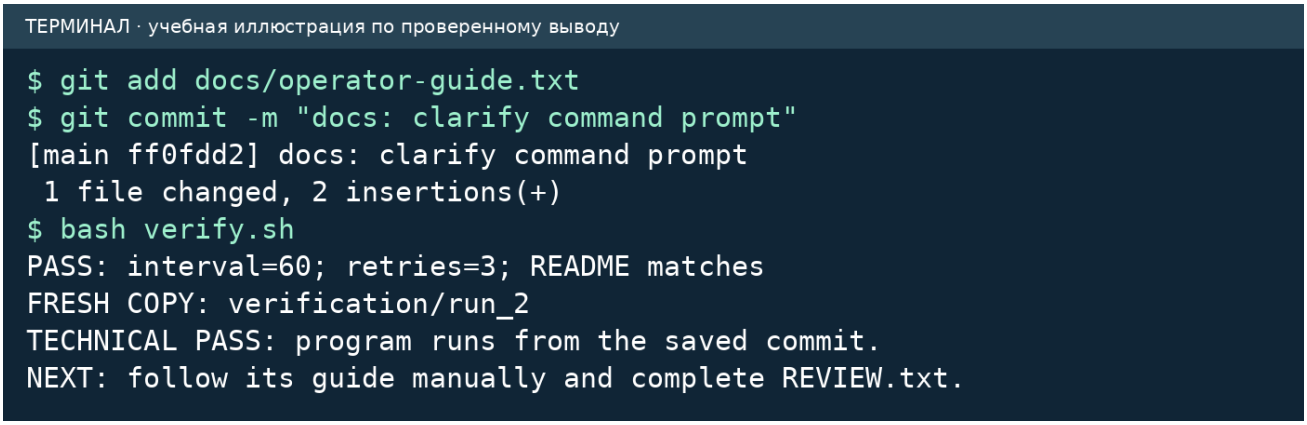


Рисунок 13. Пример второй редакции после уточнения о символе приглашения.

```
git add docs/operator-guide.txt
git commit -m "docs: clarify command prompt"
bash verify.sh
```

После исправления повторяются коммит и verify.sh. Затем снова проверьте руководство из нового каталога. В примере run_2 означает вторую попытку. Сообщение коммита в вашем упражнении должно отражать вашу реальную правку; хеш не обязан совпадать с иллюстрацией.

Заполнение REVIEW.txt

Откройте REVIEW.txt в корне operator-lab. Укажите автора, каталог свежей копии и коммит из evidence/verified_commit.txt. Заполните четыре пункта чек-листа. Опишите действие и фактический результат. Затем запишите замечание, исправление и результат повтора. Если изменений не потребовалось, объясните, какой конкретный шаг был проверен успешно.

В поле ограничений укажите, например, что проверялся только один компьютер или не проверялись другие условия. Самопроверка не заменяет оценивания преподавателем. Замените все подсказки [заполнить] и сохраните текст в UTF-8.

13 Сбор результата

Убедитесь, что последняя редакция руководства сохранена коммитом и проверена. Запустите pack.sh. Он проверяет обязательные файлы, коды ожидаемых результатов и совпадение HEAD с последним проверенным коммитом. Содержательную оценку текста выставляет преподаватель.

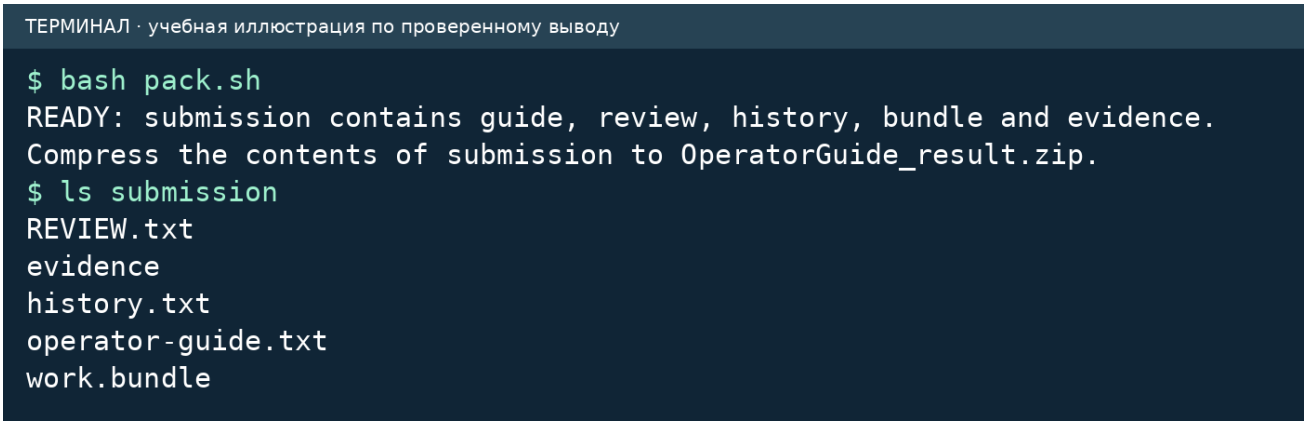


Рисунок 14. Состав папки сдачи после успешной упаковки.

```
bash pack.sh
ls submission
```

Откройте submission в Проводнике. Выделите её содержимое. В Windows 10 используйте «Отправить» и «Сжатая ZIP-папка», в Windows 11 — «Сжать в ZIP-файл» либо соответствующий пункт дополнительных параметров. Назовите архив OperatorGuide_result.zip и перенесите рядом с submission, чтобы он не оказался вложен в следующий архив.

Откройте ZIP и проверьте наличие operator-guide.txt, REVIEW.txt, history.txt, work.bundle и evidence. В evidence находятся протоколы, коды, идентификатор проверенного коммита и release.tar. Храните архив и исходную папку до завершения проверки преподавателем.

Позже загрузите один ZIP в соответствующее задание ЭОС. Ссылку и срок объявят отдельно. Если потребуется окончательная отправка после загрузки, подтвердите её кнопкой отправки задания. Отправка сообщений в мессенджере не является частью этой работы.

14 Разбор затруднений

Признак	Действие
bash или git не найдены	Проверьте, что открыли Git Bash. После установки Git закройте старое окно и откройте новое.
check.sh не найден	Выполните pwd и ls. Откройте терминал в папке, где лежит check.sh.
operator-guide.txt.txt	Включите расширения в Проводнике. Сохраните правильное имя существующего файла.
Код отказа получился 0	Повторите весь блок наблюдения. echo $? должен идти сразу после проверяемой команды.
STOP при verify.sh	Прочитайте причину: незаполненный текст, несохранённые изменения или изменение исходной программы.
pack.sh требует verify.sh	После последнего коммита повторите проверку свежей копии и актуализируйте REVIEW.txt.

Как задавать вопрос преподавателю

Сообщите номер страницы, текущую папку, точную команду и полный ответ терминала. Уточните, что ожидали увидеть. Фраза «у меня не работает» не позволяет отличить ошибку пути от неверного имени файла или пропущенного шага.

Что нельзя исправить переименованием результата

Не заменяйте FAIL на PASS вручную в протоколе и не переписывайте код 1 на 0. Отказы в трёх примерах ожидаемы и показывают, что проверка работает. Если получили другой результат, устраните причину и повторите наблюдение. Не исправляйте check.sh для обхода условий упражнения.

Если время пары закончилось, сохраните текущие файлы и отметьте незавершённый шаг. При продолжении используйте ту же рабочую папку. Начальную подготовку init.sh повторять не требуется.

15 Критерии и самостоятельная проверка

Критерий	Полное выполнение на 2 балла
Назначение и условия	Указаны границы проверки, среда, файлы, папка запуска и допустимые значения.
Последовательность запуска	Пользователь может выполнить действия; приведены команда, результат, код и завершение.
Сообщения и восстановление	Объяснены PASS и три FAIL; действия соответствуют причинам ошибок.
Проверка руководства	Есть свежая копия, заполнен чек-лист самопроверки, записаны результат и замечание либо проверенное удобство.
Версия и сдача	Руководство в коммите, проверен последний коммит, комплект сдачи полный.

За каждый критерий: 2 балла — выполнено полностью и подтверждено; 1 — выполнено частично или есть неточности; 0 — отсутствует либо противоречит результатам. Максимум 10 баллов. Дополнительное задание не заменяет обязательные пункты.

Контрольные вопросы

1. Почему одного сообщения TECHNICAL PASS недостаточно для проверки руководства?

2. Как определить, какая редакция руководства вошла в проверенный выпуск?

3. Почему три разные ошибки возвращают одинаковый код 1?

4. Что нужно повторить после исправления текста руководства?

5. Зачем при самопроверке нужен чек-лист и свежая копия?

Дополнительно для закончивших раньше

Напишите короткий раздел «Частые вопросы» из трёх вопросов на основе собственной проверки. Согласуйте ответы с основной инструкцией. Если включили дополнение в руководство, сохраните новый коммит, повторите verify.sh и обновите протокол. Это необязательное расширение.

Справка: https://git-scm.com/docs/git-archive и https://git-scm.com/docs/git-bundle. Учебная структура руководства связана с предоставленной лекцией. После загрузки результата сохраните свою рабочую папку до получения оценки.